Variablen und Datentypen

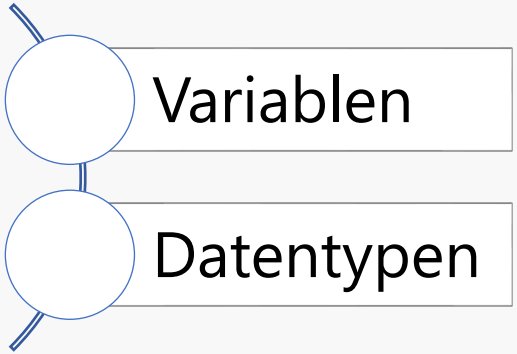
Einführung in die Programmierung 1

Wintersemester 25

Was wurde zuletzt besprochen?

- Aufbau einfacher Java-Programme
- Übersetzen und Ausführen von Java-Programmen

Was wird in diesem Foliensatz besprochen?



Was lernen Sie kennen?

- Einfachen Datentypen in Java
- Deklaration von Variablen in Java
- Zuweisung von Werten zu Variablen
- Ausgabe von Werten

Variablen

Ein Programm verarbeitet **Daten**, die in **Variablen** abgelegt werden

Ein Programm legt die **Ergebnisse** wieder in solchen **Variablen** ab

Eine Variable ist eine **benannte Speicherstelle**

Variablenbegriff

Mathematik - Platzhalter für eine unbekannte oder eine sich ändernde Zahl in Formulierungen oder Formeln der Mathematik

Informatik - Bezeichnung einer Speicherstelle, die verschiedene Werte aufnehmen kann

Variable in Java

Wird einmal vereinbart (deklariert)

- Name (Bezeichner)
- Datentyp

Beispiel

- Variable für eine ganze Zahl mit dem Namen number deklarieren

```
int number;
```

- Bedeutung
  - **int** = Datentyp (steht für ganze Zahlen)
  - number = Name (für Zugriff)

Variablen in Java – Kennzeichen

Name

Datentyp

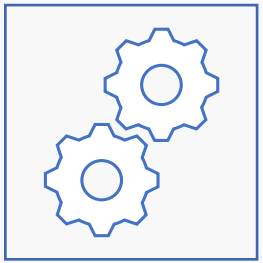
Wert

Name (Bezeichner)

- Bezeichner ist eine Folge von Zeichen
- Java-Spezifikation gibt erlaubten Aufbau vor
  - https://docs.oracle.com/javase/specs/jls/se23/html/jls-3.html#jls-3.8
  - Beispiele

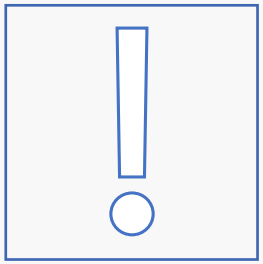
Korrekte Bezeichner	Nicht korrekte Bezeichner
counter	1counter
carWheel	car-Wheel oder car/Wheel
MAX_COUNT	while
test2	$%

Bezeichner



Compiler wird alle korrekten Bezeichner akzeptieren

Ein korrekter Bezeichner muss noch lange nicht sinnvoll sein



Bezeichner sollten lesbar und verständlich sein

Sinnvolle Bezeichner bei Variablen

Beschreibung	Negatives Beispiel	Positives Beispiel
Vollständige Wörter	c ★	counter
Kleinschreibung	COUNTER ★★	counter
Neue Teile mit Großbuchstaben („lowerCamelCase“)	firstletterintext	firstLetterInText
Sinnvolle (zweckbeschreibende) Bezeichner	$0x0l	number
Abkürzungen vermeiden	bc	byteCount
Englische Bezeichner bevorzugen	erschtaBuchstobn	firstLetter

★ Lokale Variablen, Schleifenvariablen, kleine Beispielprogramme
★★ Nur bei Konstanten üblich

Die Länge eines Bezeichners sollte der Größe des Bereichs entsprechen, in dem er verwendet wird

Einzelne Buchstaben – Oracle-Konventionen

Bezeichner	Datentyp
b	byte
c	char
d	double
e	Exception
f	float
i, j, k	int
l	long
o	Object
s	String
v	Beliebiger Wert eines Typs

https://docs.oracle.com/javase/specs/jls/se23/html/jls-6.html#jls-6.1

Datentyp

Daten haben einen bestimmten Typ (Datentyp)

Der Datentyp definiert

- Menge von **Werten**, die zu diesem Typ gehören (Wertebereich)
- Menge von **Operationen**, die mit den Werten des Typs ausgeführt werden können

Compiler kann vor der Ausführung des Programms Überprüfungen durchführen

Einfache Datentypen in Java

Ganze Zahlen	byte, short, int, long
Gleitkommazahlen	float, double
Logische Werte	boolean
Zeichen	char

Spezialfall Klasse String

String ist eine Sequenz von Zeichen

Ist kein einfacher Datentyp sondern ein Referenztyp

Wird in bestimmten Situationen wie ein einfacher Datentyp behandelt

Einfache Datentypen in Java – Details

Typ	Größe (Bits)	Werte	Standard
boolean		true oder false [Repräsentation von der JVM abhängig]	
char	16	'\u0000' bis '\uFFFF' (0 bis 65535)	(ISO Unicode character set)
byte	8	–128 bis +127 (-2^7 bis $2^7 - 1$)	
short	16	–32 768 bis +32 767 (-2^{15} bis $2^{15} - 1$)	
int	32	–2 147 483 648 bis +2 147 483 647 (-2^{31} bis $2^{31} - 1$)	
long	64	–9 223 372 036 854 775 808 bis +9 223 372 036 854 775 807 (-2^{63} bis $2^{63} - 1$)	
float	32	Negativer Bereich: –3,4028234663852886e+38 bis –1,40129846432481707e–45 Positiver Bereich: 1,40129846432481707e–45 bis 3,4028234663852886e+38	(IEEE 754 floating point)
double	64	Negativer Bereich: –1,7976931348623157e+308 bis –4,94065645841246544e–324 Positiver Bereich: 4,94065645841246544e–324 bis 1,7976931348623157e+308	(IEEE 754 floating point)

Literal

- Direkte Darstellung von Werten in Programmen
- Unterschiedliche Darstellungsmöglichkeiten
  - Z. B. dezimal, hexadezimal, oktal, binär für ganze Zahlen
- Aufbau

https://docs.oracle.com/javase/specs/jls/se23/html/jls-3.html#jls-3.10

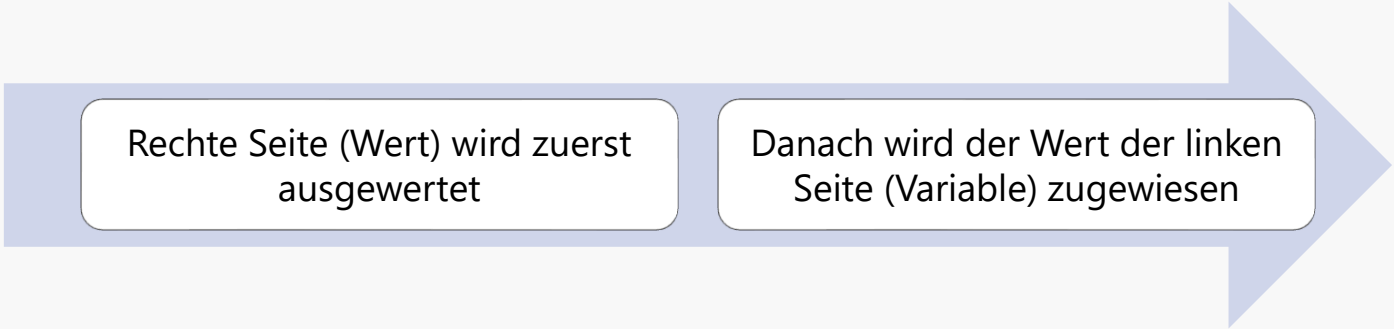
Beispiele
• int: 7482, 123_456, -234, 012, 0x3F, 0b11011 • double: 24.75, -0.3423, 2., .3, 1.34e2 • boolean: true, false • char: 'a', '1', '%', '\n' • String: "Hello", "Input:"

Wert zuweisen (1)

- Form
  - `Variable = Wert;`
- Aufbau
  - Links steht eine Variable
  - Rechts kann z. B. ein fixer Wert (Literal), eine Variable, eine Berechnung etc. stehen

Wert zuweisen (2)

- Ablauf (Aktion)
  - `Variable = Wert;`



Deklaration und Initialisierung (1)

- Deklaration mit Initialisierung

```
int number = 10;
```

- Einfache Deklaration, danach Zuweisung (ohne Datentyp)

```
int number;
…
number = 10;
```

Deklaration und Initialisierung (2)

- Deklaration mit Initialisierung, Deklaration mit Zuweisung aus anderer Variable bzw. einem Ausdruck

```
int number = 10;
int number2 = number;
int number3 = number * 42 + 23;
```

Ausdruck (Verknüpfung von Variablen und/oder Literalen durch Operatoren)

- Hinweis
  - Vor der ersten Verwendung auf der rechten Seite **muss** eine Variable einen Wert erhalten (entweder explizit oder automatisch initialisiert)

4 typische Szenarien von Zuweisungen

Wert zuweisen

- Z.B. number = 42;

Ergebnis eines Ausdrucks zuweisen

- Z.B. number = 3 * factor;

Duplizieren von Werten

- Z.B. number = value;

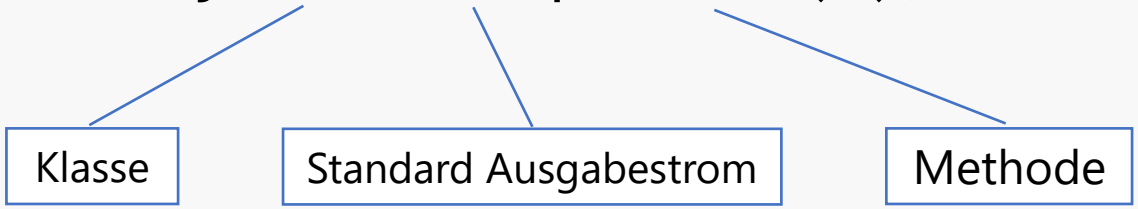
Akkumulieren

- Z.B. number = number + 1;

Ausgabe von Werten

- Ausgabe
  - Muss nicht selbst programmiert werden
  - Wiederverwendung von Bibliotheksmethoden aus der Java-API (Application Programming Interface)
- Aufbau

```
System.out.println(…);
```



Ausgabe von Werten - Beispiele

```
System.out.print(4);           // Gibt 4 aus
System.out.print("Hello");     // Gibt den Text Hello aus
System.out.print(a);           // Gibt den Wert der Variable a aus
System.out.println(a);         // Gibt den Wert der Variable a aus und springt in die nächste Zeile
System.out.println();          // Gibt eine Leere Zeile aus
System.out.print(4 + 2);       // Gibt 6 aus (Ausdruck wird vor der Ausgabe ausgewertet)
System.out.print("Hello" + 4); // Gibt Hello4 aus(konkateniert die Zeichen)
```

Kommentare dienen zum Beschreiben des Codes und werden nicht übersetzt

Kommentare in Java

Einzeilig

```
// Prints numbers …
```

Mehrzeilig

```
/*
Prints
numbers
…
*/
```

Dokumentation

```
/**
This method …
*/
```

Beispiel (Deklarationen)

```java
public class Declarations {

    public static void main(String[] args) {
        int number = 145;
        long largeNumber = 100000000000000L;
        float weight = 75.5f;
        double heavyWeight = 2534.25;
        char input = 'a';
        boolean check = true;
        System.out.println(number + " " +
                largeNumber + " " +
                weight + " " +
                heavyWeight + " " +
                input + " " +
                check);
    }
}
```

```
145 100000000000000 75.5 2534.25 a true
```

final-Variablen

- Deklaration mit Initialisierung (danach keine Zuweisung möglich!)

```
final int x = 10;
final double d = 10.0;
```

- Aufgeschobene Initialisierung

```
final int x;
…
…
x = 10;
```

Konstanten

- Deklaration außerhalb von main mit Initialisierung

```
final static int VAL = 10;
```

- Können nicht mehr verändert werden

Aufzählung (Enumeration)

- Mit dem Schlüsselwort `enum` wird eine Aufzählung von Konstanten definiert
- Realisiert einen eingeschränkten Wertebereich
- Beispiel

```
enum Color {GREEN, ORANGE, RED}
```

Enumeration – Vorteile

Es müssen keine Werte explizit den Konstanten zugewiesen werden
• Compiler erzeugt selbständig eindeutige Werte

Bei Zuweisungen und Vergleichen erfolgt eine Typüberprüfung
• Falsche Werte können gar nicht zugewiesen werden

Enumeration (Beispiel)

Datentyp Color mit drei möglichen Werten

```
public class EnumTest {
    enum Color {GREEN, ORANGE, RED}

    public static void main(String[] args) {
        Color color1 = Color.RED;
        Color color2 = Color.ORANGE;
        Color color3 = Color.GREEN;
        System.out.println(color1 + " " + color2 + " " + color3);
    }

}
```

```
RED ORANGE GREEN
```